

ICT2203

Network Security



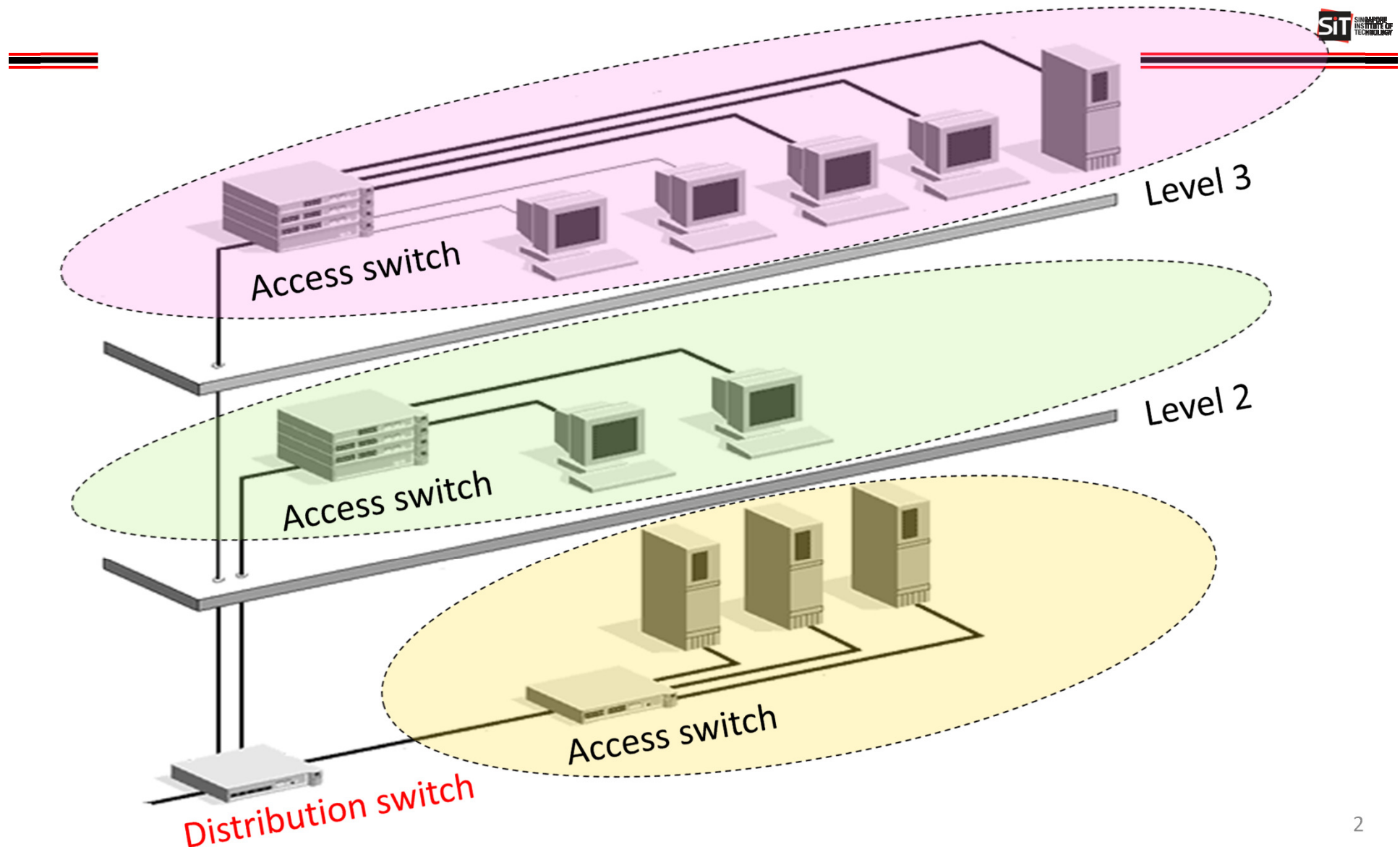
Lab 1 Notes:

Attacks and Defense of Local Area Networks (LAN) with Switches Part 1

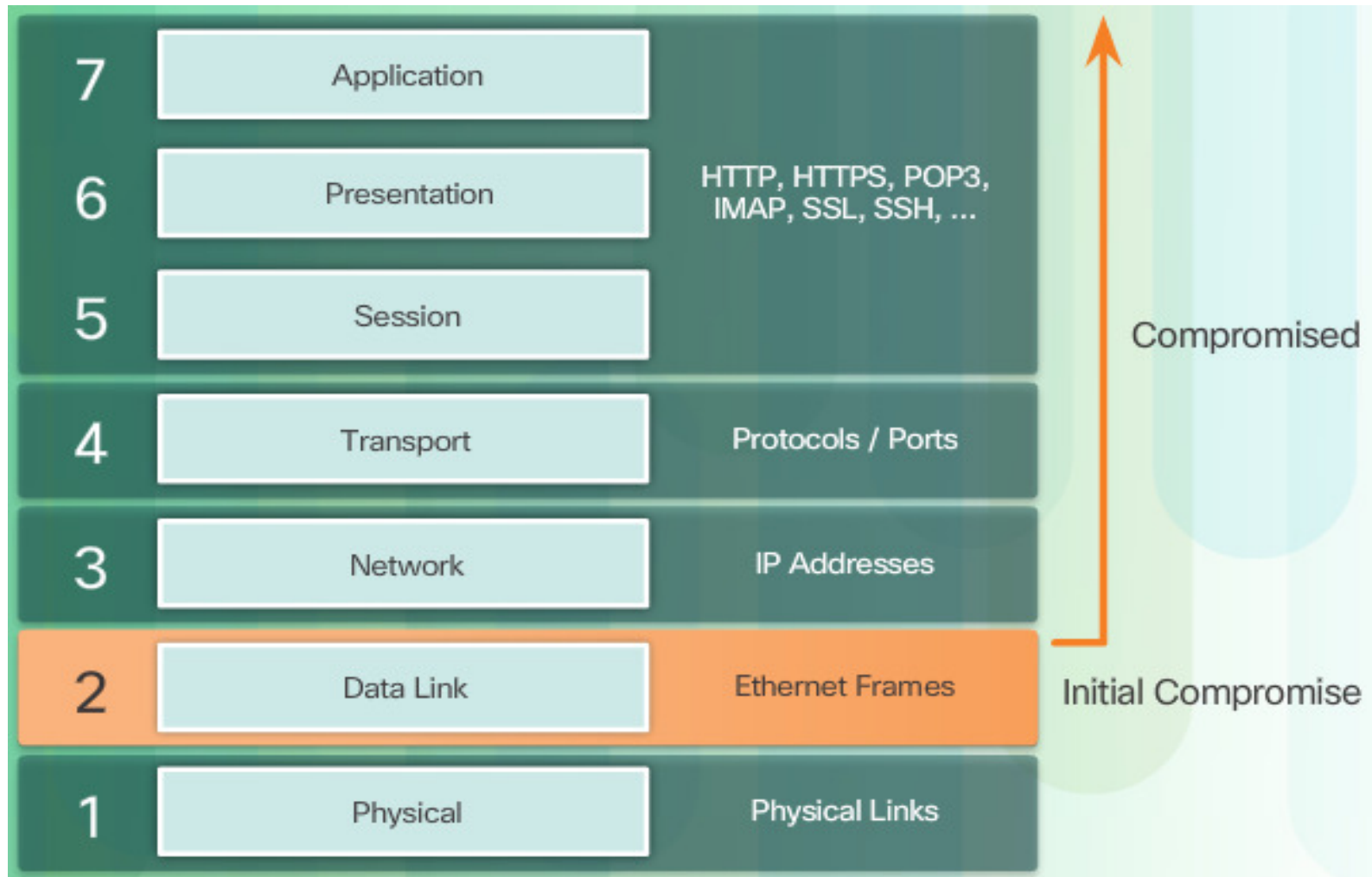
2021-2022 Trimester 1



In real world networks, the first **entry point** for (insider) **attackers** are usually the **LAN switches**, or **access switches**.



Hence, it's **important** to **secure LAN switches** because compromising layer 2 will compromise layer 3 and above which are encapsulated within it.



Lab Exercise 1

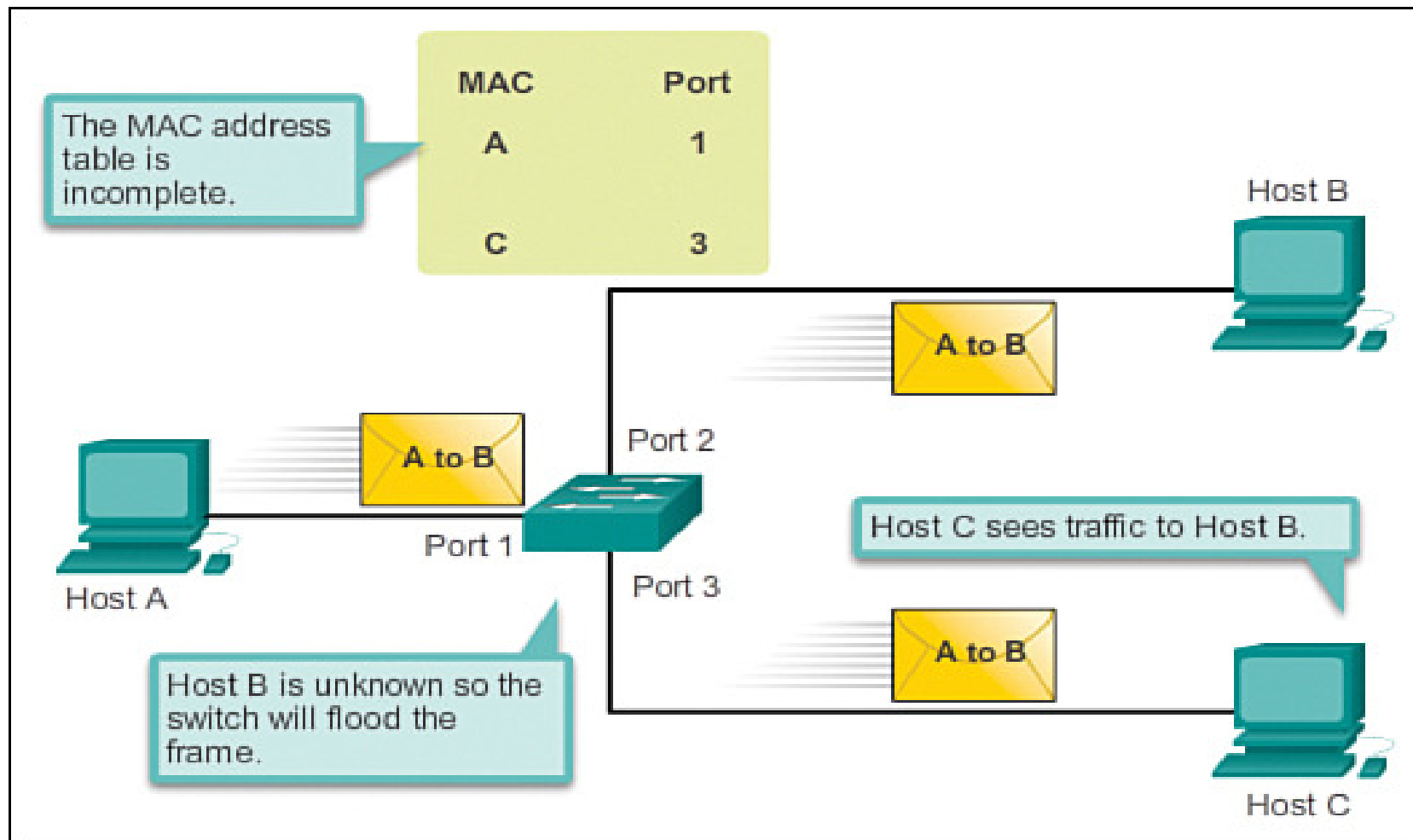
1.1 MAC flooding attack

1.2

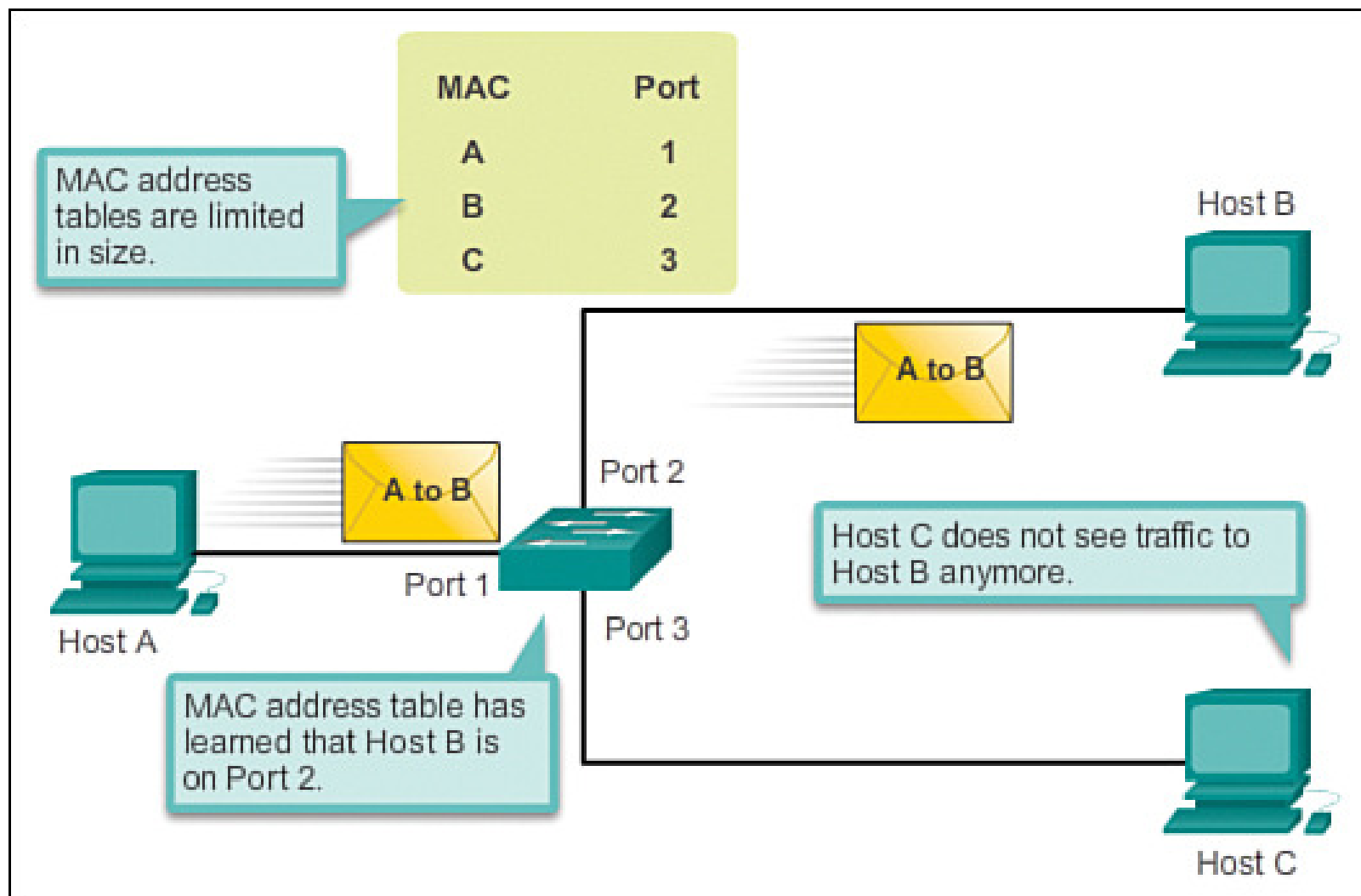
Defense:

- Enable **port security** in switches

Recall from ICT1010 about the [switch's MAC address table](#). If the [destination MAC](#) address of a frame is [not found](#) in the table, it will be [flooded](#) out to all ports except incoming port.



After **address self-learning**, a unicast frame will only be forwarded out to the **intended port**, and not others. Thus, a **switch** is considered to be more 'secure' than a **hub**.

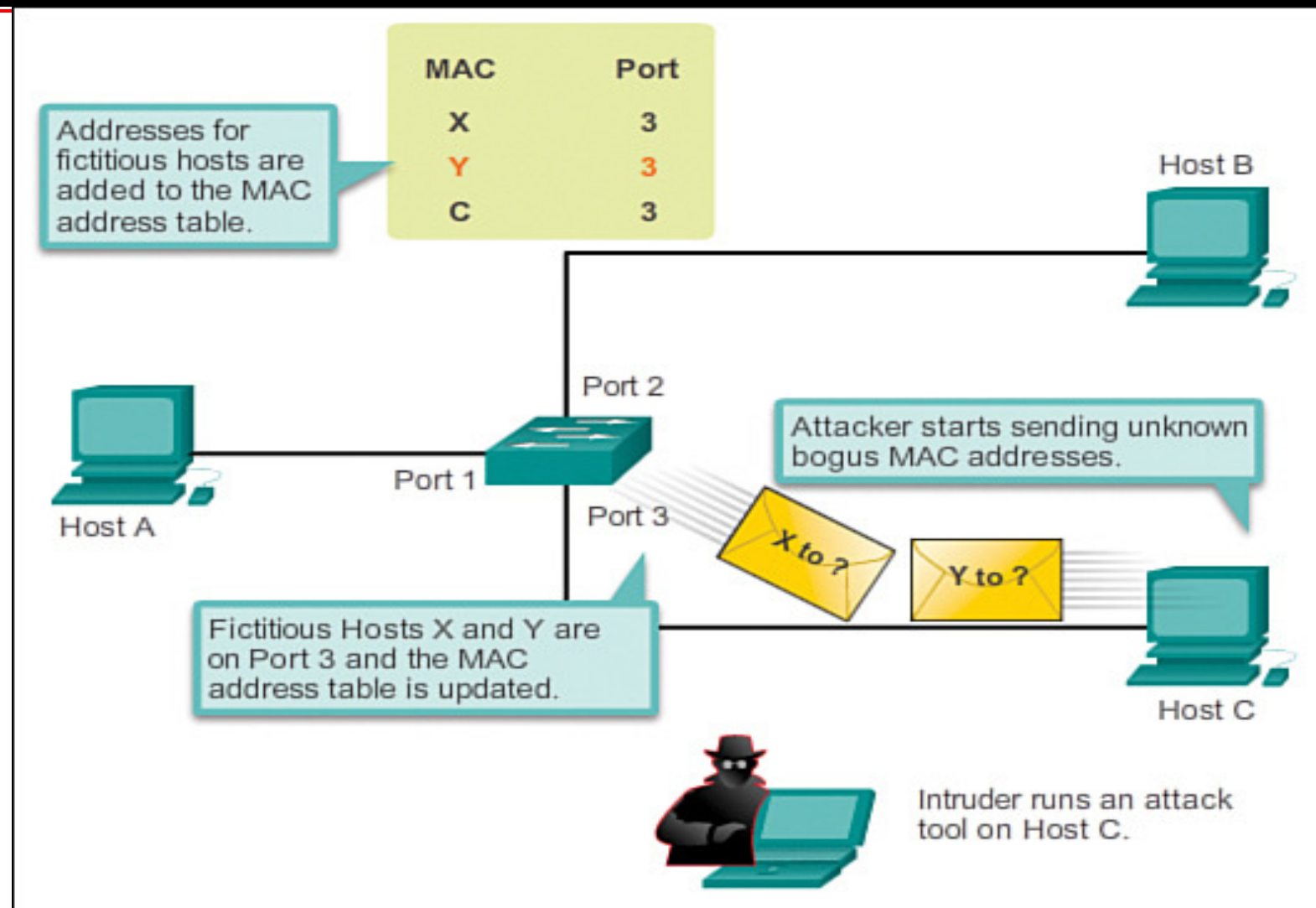


However, the switch's **MAC address table** cannot be of infinite size. What happens when a switch does not have sufficient memory to store more new MAC addresses?

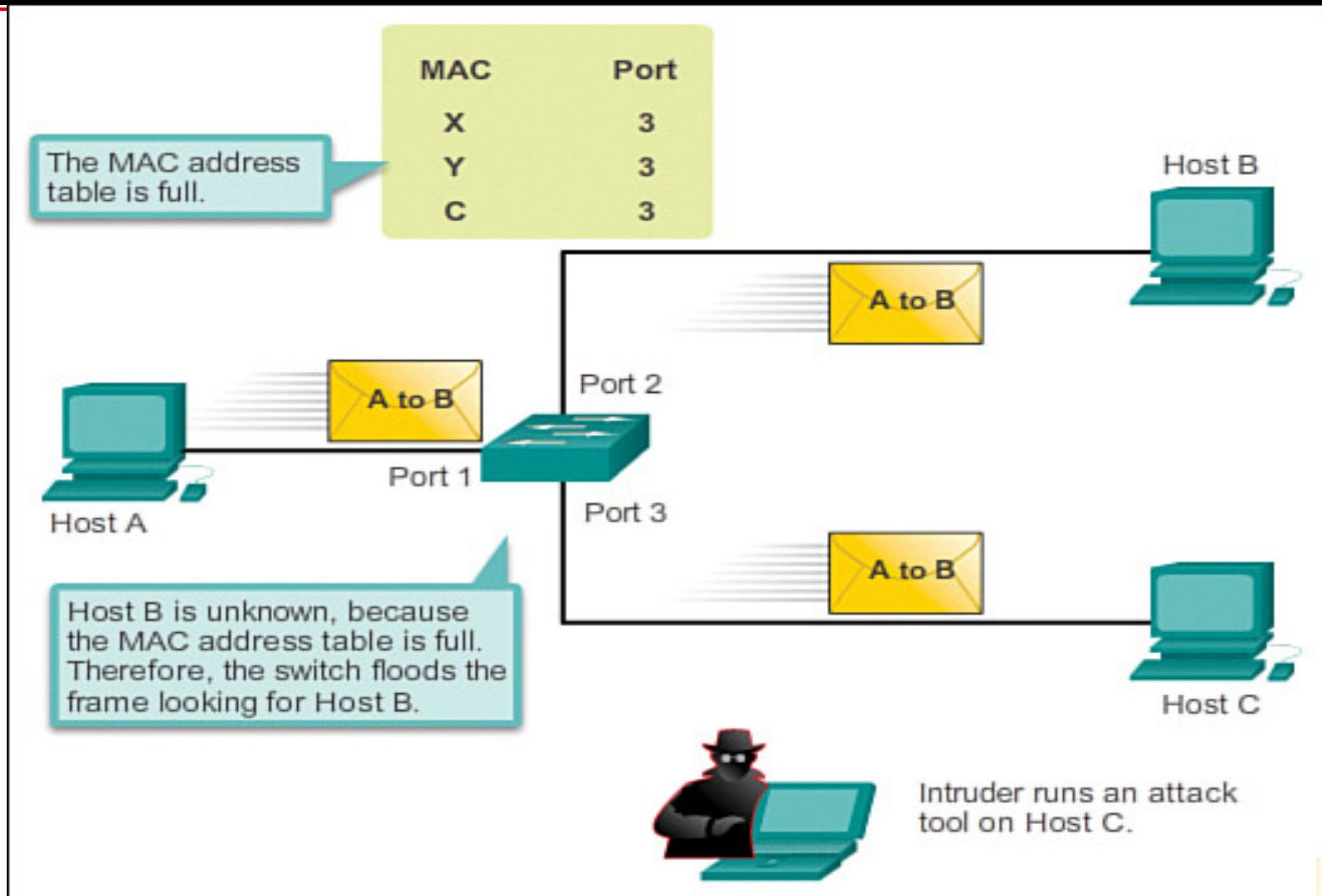
E.g.: The address table size of various models of Cisco switches

Switch Model	Number of Bridge-Table Entries
Cisco Catalyst Express 500	8000
Cisco Catalyst 2948G	16,000
Cisco Catalyst 2940/50/55/60/70	Up to 8000
Cisco Catalyst 3500XL	8192
Cisco Catalyst 3550/60	Up to 12,000 (depending on the model)
Cisco Catalyst 3750/3750M	12,000
Cisco Catalyst 4500	32,768
Cisco Catalyst 4948	55,000
Cisco Catalyst 6500/7600	Up to 131,072 (more if distributed feature cards are installed)

MAC flooding attack: bombarding the switch with frames containing **fake source MAC addresses** until the MAC address table is full.



As a result, the **destination address** of an authentic frame is likely **not to be found** in the **address table**, and it will be **flooded** and **received by attackers**!



A common tool for conducting **MAC flooding attack** is **macof** (in Kali Linux) which sends out frames with randomly generated source and destination MAC addresses.

Refer to <http://linux.die.net/man/8/macof> on the use of macof:

```
[root@client root]# macof -n 5

3a:50:db:3f:e9:c2 75:83:21:6a:ca:f 0.0.0.0.30571 > 0.0.0.0.19886: S
212769628:212769628(0) win 512
db:ad:aa:2d:ac:e9 f6:fe:a7:25:4b:9a 0.0.0.0.4842 > 0.0.0.0.13175: S
1354722674:1354722674(0) win 512
2b:e:b:46:a8:50 d9:9e:bf:1f:8f:9f 0.0.0.0.32533 > 0.0.0.0.29366: S
1283833321:1283833321(0) win 512
ce:56:ee:19:85:1a 39:56:a8:38:52:de 0.0.0.0.26508 > 0.0.0.0.8634: S
886470327:886470327(0) win 512
89:63:d:a:13:87 55:9b:ef:5d:34:92 0.0.0.0.54679 > 0.0.0.0.46152: S
1851212987:1851212987(0) win 512
[root@client root]#
```

Far-Left/High-Order-Octet	Value in Binary
2B	0010 101 1
DB	1101 101 1
89	1000 100 1

Note that **multicast** addresses (refer ICT1010 Lec 2) will not be learned by the switch.

Here's an example of the switch's address table after **macof attack** in previous slide.

MAC address table **before attack**:



```
6K-1-720# sh mac address-table dynamic vlan 20
Legend: * - primary entry
        age - seconds since last seen
        n/a - not available
```

	vlan	mac address	type	learn	age	ports
*	20	00ff.01ff.01ff	dynamic	Yes	45	Gi1/15

MAC address table **after attack**:

```
6K-1-720# sh mac address-table dynamic vlan 20
Legend: * - primary entry
        age - seconds since last seen
        n/a - not available
```

	vlan	mac address	type	learn	age	ports
*	20	ce56.ee19.851a	dynamic	Yes	70	Gi1/15
*	20	00ff.01ff.01ff	dynamic	Yes	70	Gi1/15
*	20	3a50.db3f.e9c2	dynamic	Yes	70	Gi1/15

Note that the entries in the MAC address table will be removed automatically if not seen again within the **aging time**, default to **300 seconds**.

To **configure aging time** for entries in MAC address table:

```
switch(config)# mac address-table aging-time seconds [ vlan  
vlan-id ]
```

seconds : range from 10 to 1000000 seconds

To **return** to default setting:

```
switch(config)# no mac address-table aging-time
```

To **disable aging time**:

```
switch(config)# mac address-table aging-time 0 [ vlan vlan-  
id ]
```

So to be successful in **MAC flooding attack**, attackers will need to flood sufficient frames within the aging time.

Count of MAC address table entries **before attack**:

```
6K-1-720# clear mac address-table dynamic
MAC entries cleared.
```

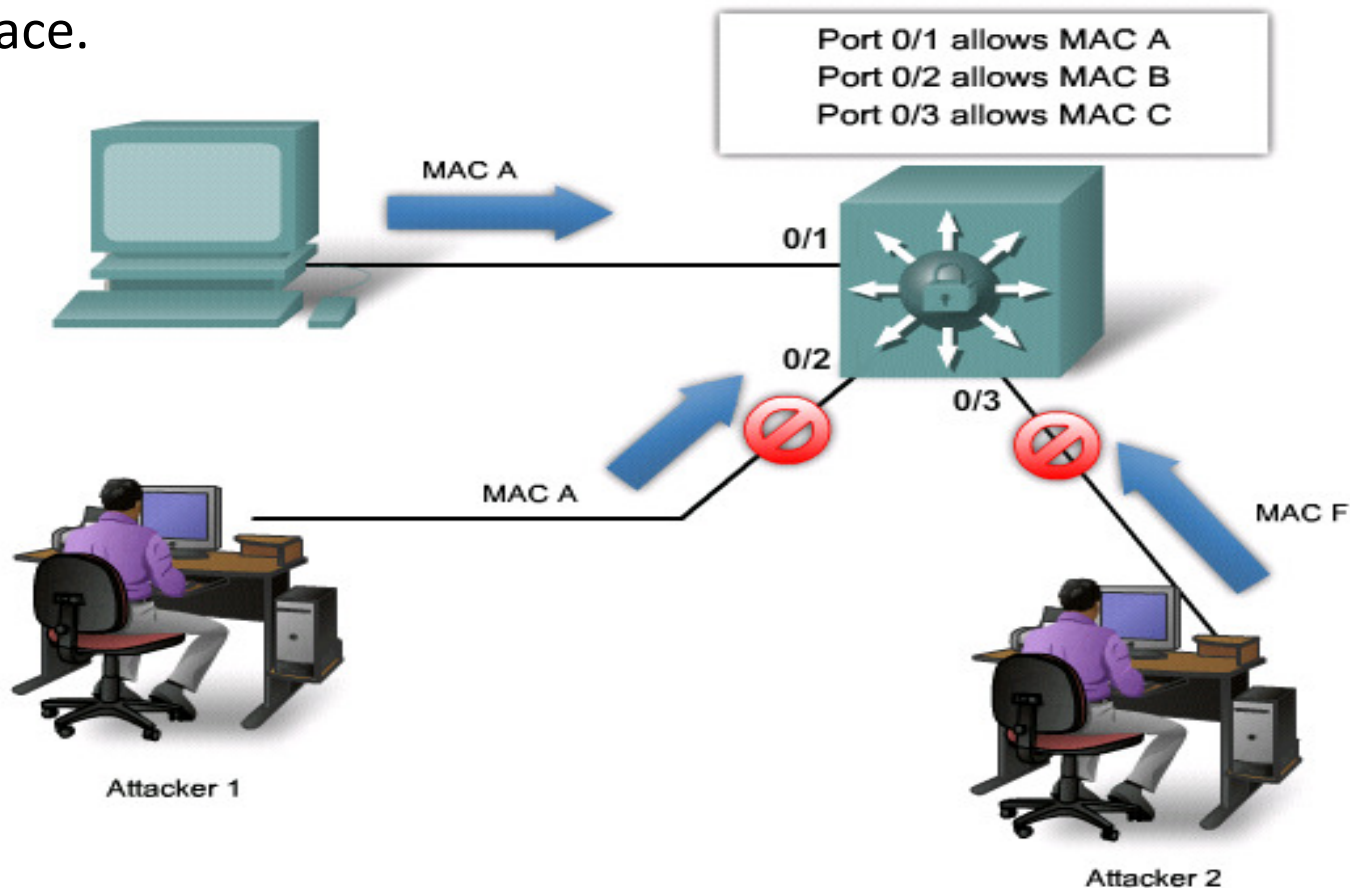
```
6K-1-720# show mac address-table count
MAC Entries for all vlans :
Dynamic Address Count:          37
Static Address (User-defined) Count: 494
Total MAC Addresses In Use:      531
Total MAC Addresses Available:   65536
```

Count of MAC address table entries **after attack**:

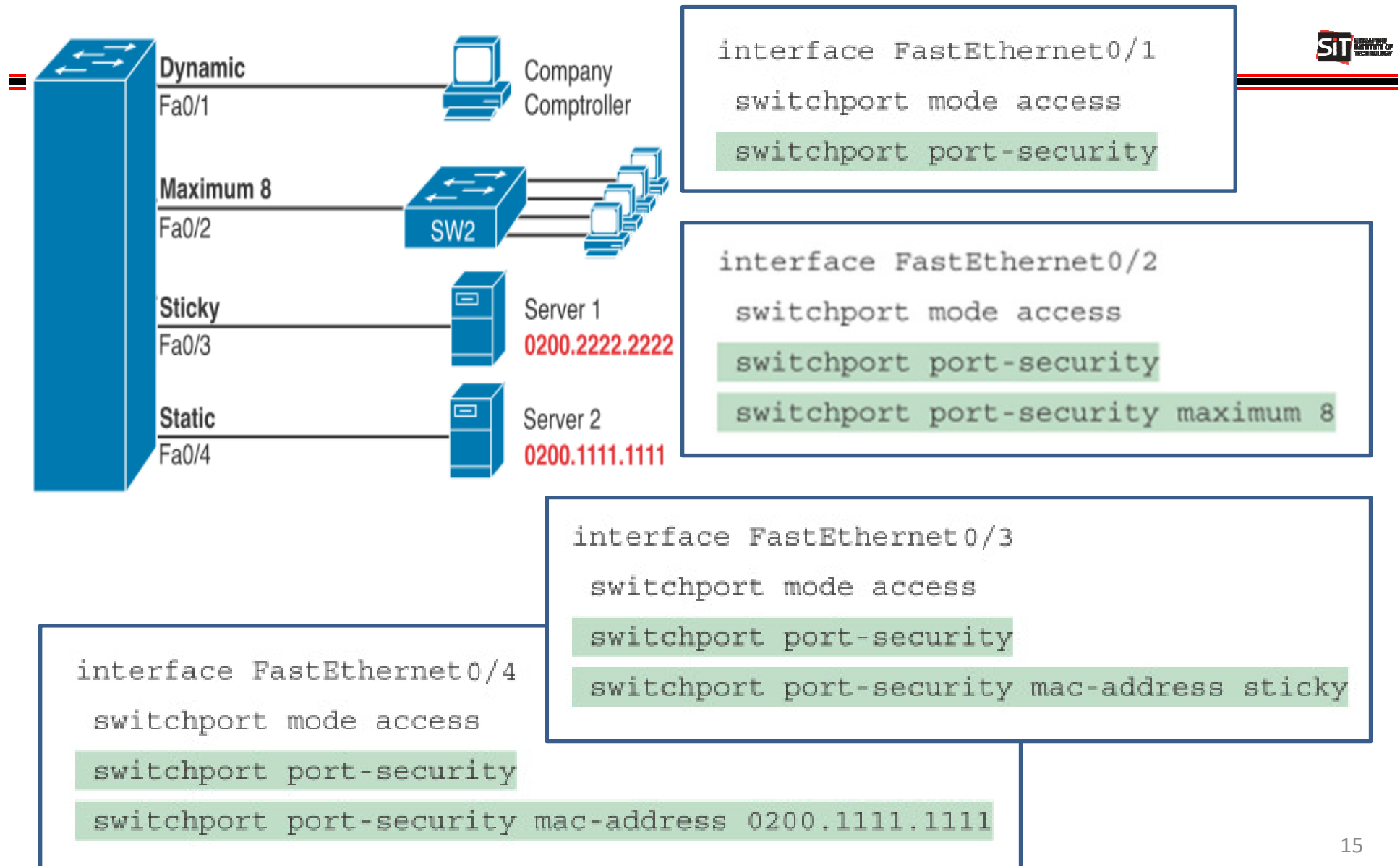
```
6K-1-720# show mac address-table count
MAC Entries for all vlans :
Dynamic Address Count:          58224
Static Address (User-defined) Count: 503
Total MAC Addresses In Use:      58727
Total MAC Addresses Available:   65536
```

To prevent MAC flooding attack: enable Port Security in switches.

Port security restricts input to a switch's interface by identifying and limiting the MAC addresses of the hosts that are allowed to access the interface.



In Cisco switches, **port security** may be configured with one or more of the following options:



Explanation of the **port-security** commands:

Command	Mode/Purpose/Description
<code>switchport port-security mac-address <i>mac-address</i></code>	Interface configuration mode command that statically adds a specific MAC address as an allowed MAC address on the interface.
<code>switchport port-security mac-address sticky</code>	Interface subcommand that tells the switch to learn MAC addresses on the interface and add them to the configuration for the interface as secure MAC addresses.
<code>switchport port-security maximum <i>value</i></code>	Interface subcommand that sets the maximum number of static secure MAC addresses that can be assigned to a single interface.

To verify port-security configuration, use the `show port-security` or `show port-security interface` commands as follows:

```
SW1# show port-security interface fastEthernet 0/3
Port Security           : Enabled
Port Status             : Secure-up
Violation Mode          : Shutdown
Aging Time              : 0 mins
Aging Type              : Absolute
SecureStatic Address Aging : Disabled
Maximum MAC Addresses   : 1
Total MAC Addresses     : 1
Configured MAC Addresses : 1
Sticky MAC Addresses    : 1
Last Source Address:Vlan : 0200.2222.2222:1
Security Violation Count : 0
```

In addition, there are 3 modes to configure the actions to take if port security violation is detected.

Command	Mode/Purpose/Description
switchport port-security violation {protect restrict shutdown}	Interface subcommand that tells the switch what to do if an inappropriate MAC address tries to access the network through a secure switch port.

Option on the switchport port-security violation Command	Protect	Restrict	Shutdown*
Discards offending traffic	Yes	Yes	Yes
Sends log and SNMP messages	No	Yes	Yes
Disables the interface, discarding all traffic	No	No	Yes

*Shutdown is the default mode

For port security violation **shutdown** mode, the port will be moved to **error disabled state** if **violation** is detected.

Line Status	Protocol Status	Interface Status	Typical Root Cause
Down	Down (err-disabled)	err-disabled	Port security has disabled the interface.

Two ways to recover err-disabled ports:

1. **Manual recovery** (more control)

```
Switch(config)# interface type num/num
Switch(config-if)# shutdown
Switch(config-if)# no shutdown
```

2. **Auto recovery** (to reduce administrative overhead)

```
Switch(config)# errdisable recovery cause psecure-violation
Switch(config)# errdisable recovery interval 600
```

type

in seconds

Lab Exercise 2

2.1 Switch spoofing attack

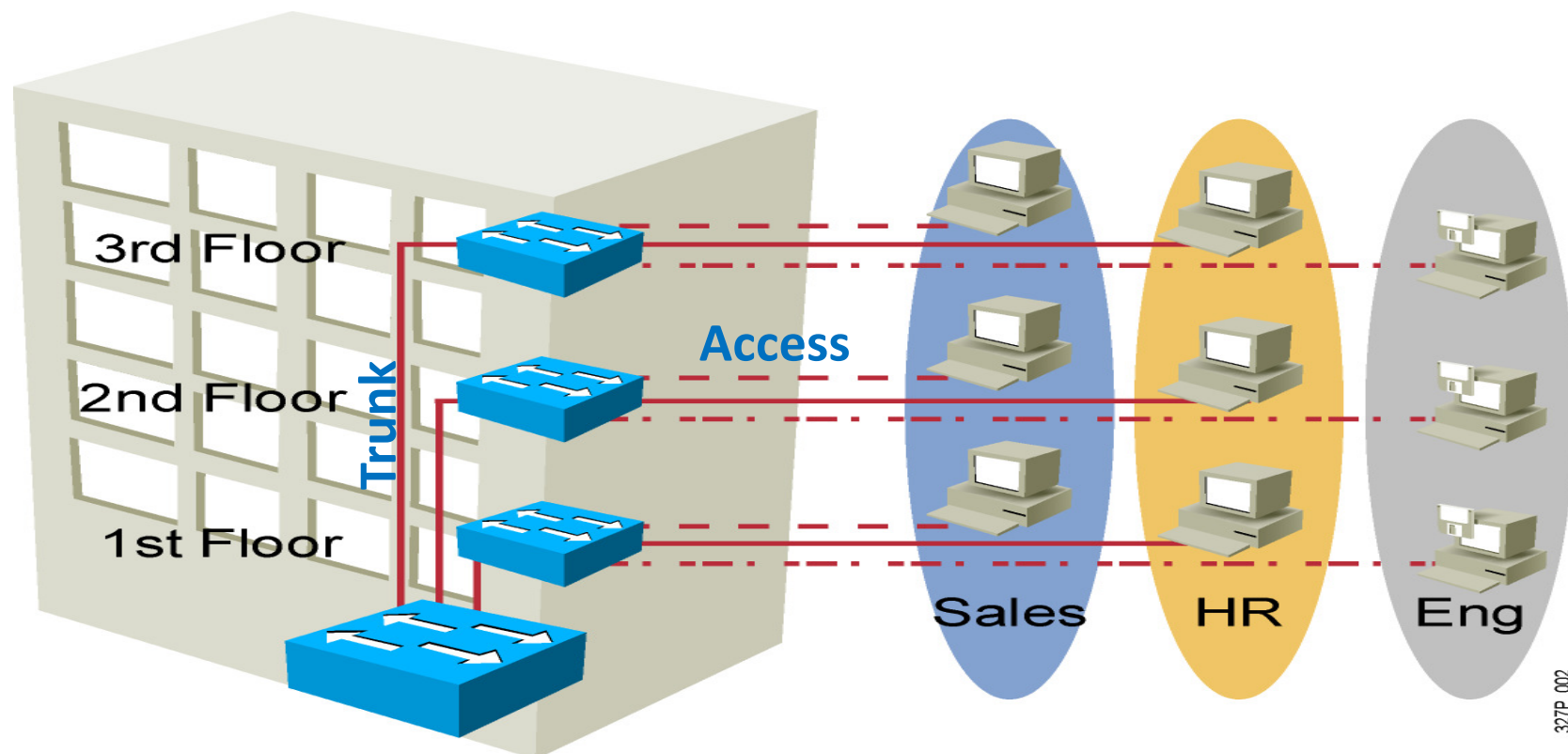
Defenses:

- 2.2 • **Manually configure** ports as access or trunk
- **Disable DTP**
- Good practice to shut down all unused ports and configure them into an unused VLAN

In ICT1010, you have learned about configuring **VLANs** (virtual LANs) on **switches** to separate frames in different VLANs to support users in different departments.

With VLANs, switch port connected to a link is operating either as:

- (i) **Access** – carry frames for **only one VLAN** configured on the port; or
- (ii) **Trunk** – carry frames for **multiple VLANs** on one physical link



To simplify configuration, modern switches are commonly shipped with mechanisms or protocols to automate trunk formation; e.g. the Cisco **Dynamic Trunking Protocol (DTP)**.



In Cisco switches, each **switch port** can be configured in 4 different modes: **access**, **dynamic auto**, **trunk** and **dynamic desirable**.

Then, DTP will be run automatically which will result in the **link** being an **access** or **trunk** as follows:

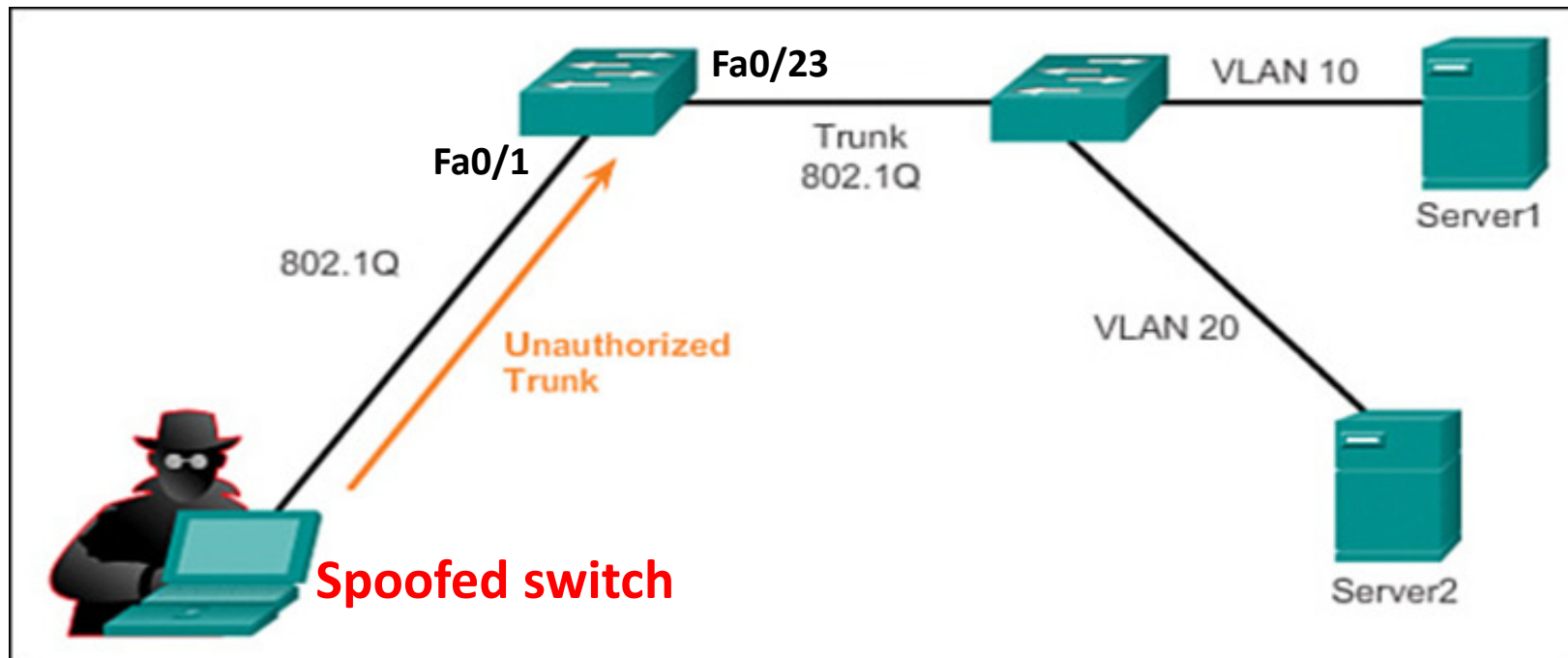
Administrative Mode	Access	Dynamic Auto	Trunk	Dynamic Desirable
access	Access	Access	Do Not Use ¹	Access
dynamic auto	Access	Access	Trunk	Trunk
trunk	Do Not Use ¹	Trunk	Trunk	Trunk
dynamic desirable	Access	Trunk	Trunk	Trunk

By **default**, Cisco switch ports are shipped in **dynamic auto** mode which will passively wait to receive trunk negotiation to become a trunk.

Unfortunately, the use of DTP can lead to **switch spoofing attack** – a form of **VLAN hopping attack** where attackers could access frames in other VLANs that normally would not be accessible.



Taking advantage of the default configuration of the switch port which is dynamic auto, an attacker can inject DTP messages to form a trunk.



With the **spoofed switch**, the attacker may now **receive frames** from other VLANs, or **send spoofed frames** tagged for any target VLAN.

To demonstrate **switch spoofing attack**, you may use the tool **Yersinia**, which is available in Kali Linux.



■ Launching Yersinia in interactive mode:

```
kali@kali: ~  
File Actions Edit View Help  
—(kali@kali)-[~]  
$ sudo yersinia -I
```

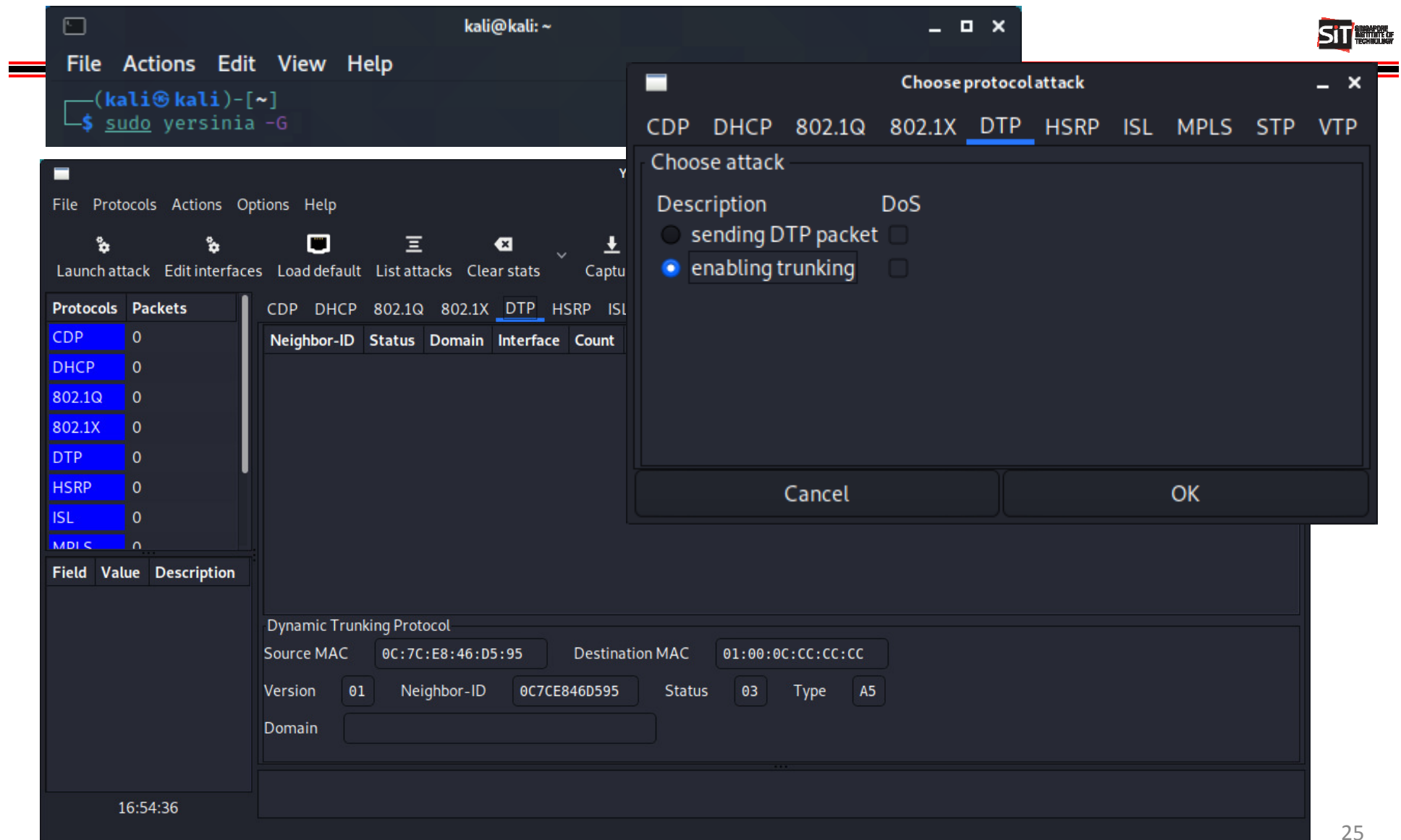
```
kali@kali: ~  
File Actions Edit View Help  
yersinia 0.8.2 by Slay & tomac - STP mode [16:44:24]  
RootId BridgeId Port Iface Last seen  
  
Total Packets: 0 STP Packets: 0 MAC Spoofing [X]  
  
STP Fields  
Source MAC 0A:23:16:02:FF:08 Destination MAC 01:80:C2:00:00:00  
Id 0000 Ver 00 Type 00 Flags 00 RootId 5080.760F0E14AC58 Pathcost 00000000  
BridgeId CB09.E7CD90117CAA Port 8002 Age 0000 Max 0014 Hello 0002 Fwd 000F
```

Press function keys to move around different attacks.

```
kali@kali: ~  
File Actions Edit View Help  
yersinia 0.8.2 Available commands [16:44:50]  
RootId h Help screen  
x eXecute attack  
i edit Interfaces  
ENTER information about selected item  
v View hex packet dump  
d load protocol Default values  
e Edit packet fields  
f list capture Files  
s Save packets from protocol  
S Save packets from all protocols  
L Learn packet from network  
M set Mac spoofing on/off  
l List running attacks  
K Kill all running attacks  
c Clear current protocol stats  
C Clear all protocols stats  
g Go to other protocol screen  
Ctrl-L redraw screen  
w Write configuration file  
a About this proggie  
q Quit (bring da noise)  
  
Total Packets: 0  
This is the help s  
STP Fields  
Source MAC 0A:23:  
Id 0000 Ver 00 Ty  
BridgeId CB09.E7C  
ce Last seen  
MAC Spoofing [X]  
:00  
thcost 00000000  
o 0002 Fwd 000F
```

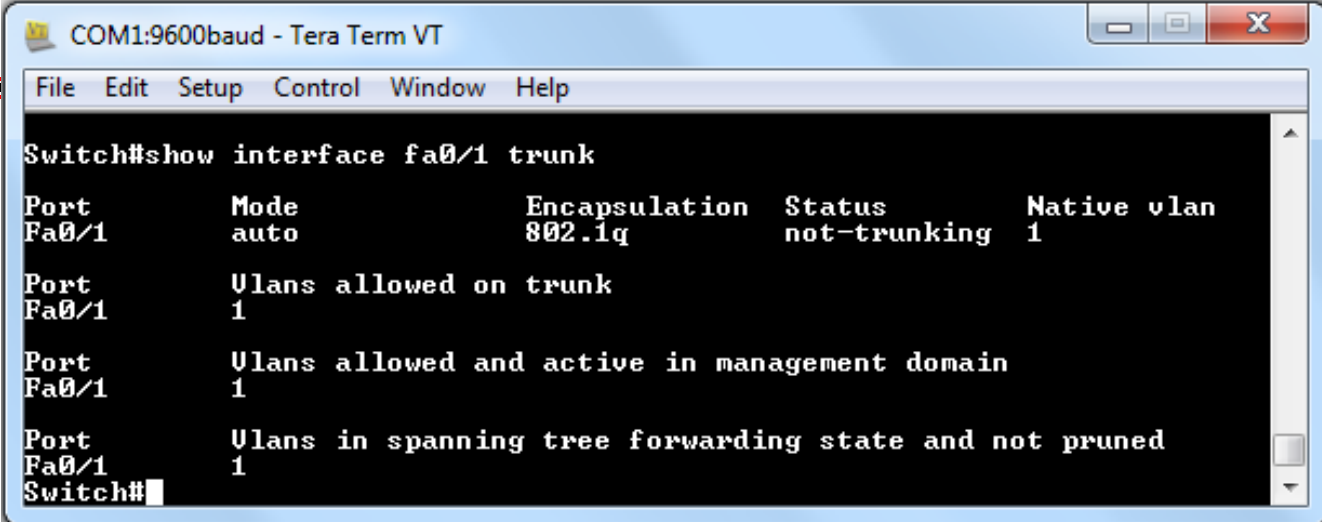
Press appropriate alphabet keys, e.g. 'h' to show help screen with available commands.

Yersinia may also be launched in graphical mode but it is only in alpha version and may not be stable.



An example of the result of conducting **switch spoofing attack** on a switch using **Yersinia**.

Port Fa0/1 in
default
dynamic auto
before attack:



```
COM1:9600baud - Tera Term VT
File Edit Setup Control Window Help

Switch#show interface fa0/1 trunk

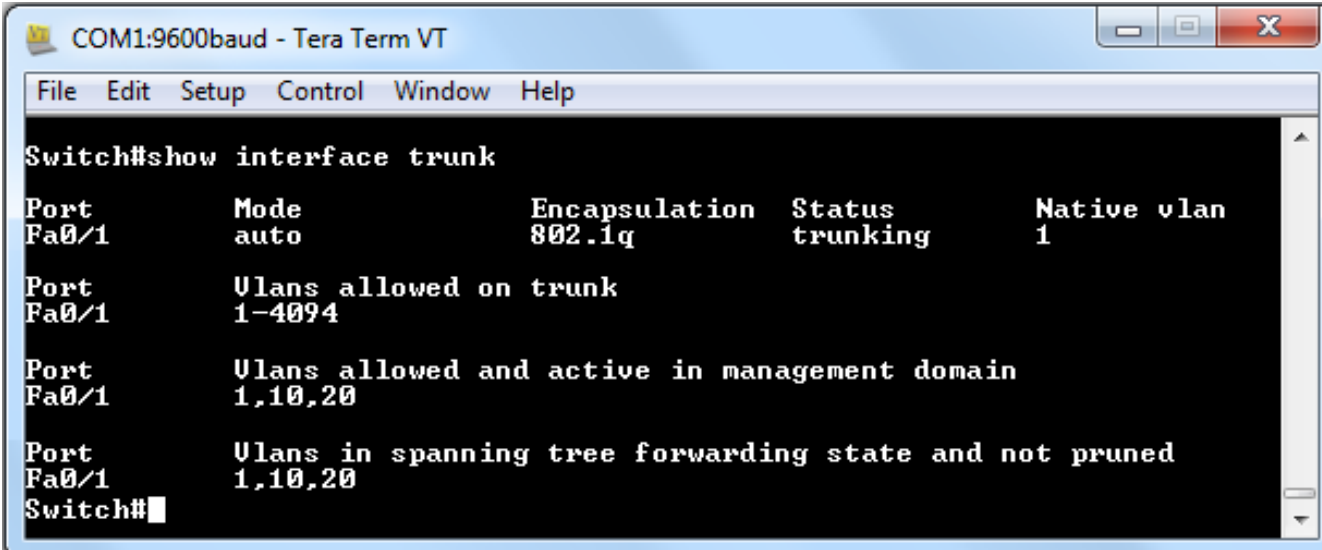
Port      Mode      Encapsulation  Status      Native vlan
Fa0/1     auto      802.1q         not-trunking 1

Port      Vlans allowed on trunk
Fa0/1     1

Port      Vlans allowed and active in management domain
Fa0/1     1

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/1     1
Switch#
```

Port Fa0/1
becomes trunk
after attack:



```
COM1:9600baud - Tera Term VT
File Edit Setup Control Window Help

Switch#show interface trunk

Port      Mode      Encapsulation  Status      Native vlan
Fa0/1     auto      802.1q         trunking     1

Port      Vlans allowed on trunk
Fa0/1     1-4094

Port      Vlans allowed and active in management domain
Fa0/1     1,10,20

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/1     1,10,20
Switch#
```

To prevent switch spoofing attack, disable DTP negotiations and manually configure ports as access or trunk explicitly.



```
SW2(config)# interface fa0/1
! Specifies that this is an access port
SW2(config-if)# switchport mode access
SW2(config-if)# switchport access VLAN 10
! Note, even for an interface configured as an access port,
! negotiation packets are still being sent unless we disable it
SW2(config-if)# switchport nonegotiate

SW2(config-if)# interface fa0/23
! Specifies the port as a trunk
SW2(config-if)# switchport mode trunk
SW2(config-if)# switchport nonegotiate
```

For added security, it is also wise to **shutdown all unused ports** and place them in an **unused VLAN** , e.g. 100, so as to prevent someone from discovering a live port that might be exploited.



For example, if **port Fa0/2** is **unused**, then **shut it down** to **prevent attacker** from discovering and exploiting it.

